

CS3520
Computer Organisation and Architecture I
Lecture 0 – Introduction

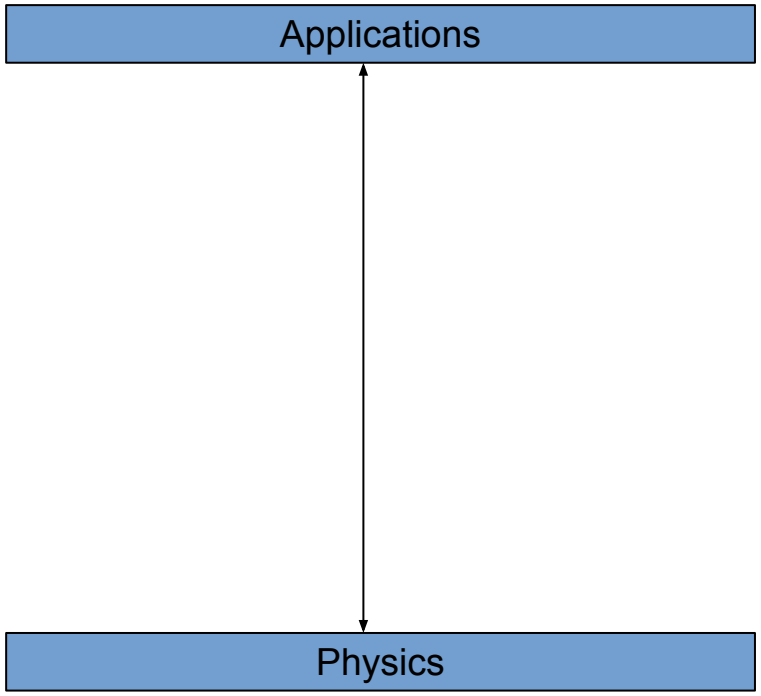
Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: khobatha.setetemela@gmail.com

Lecture Outline

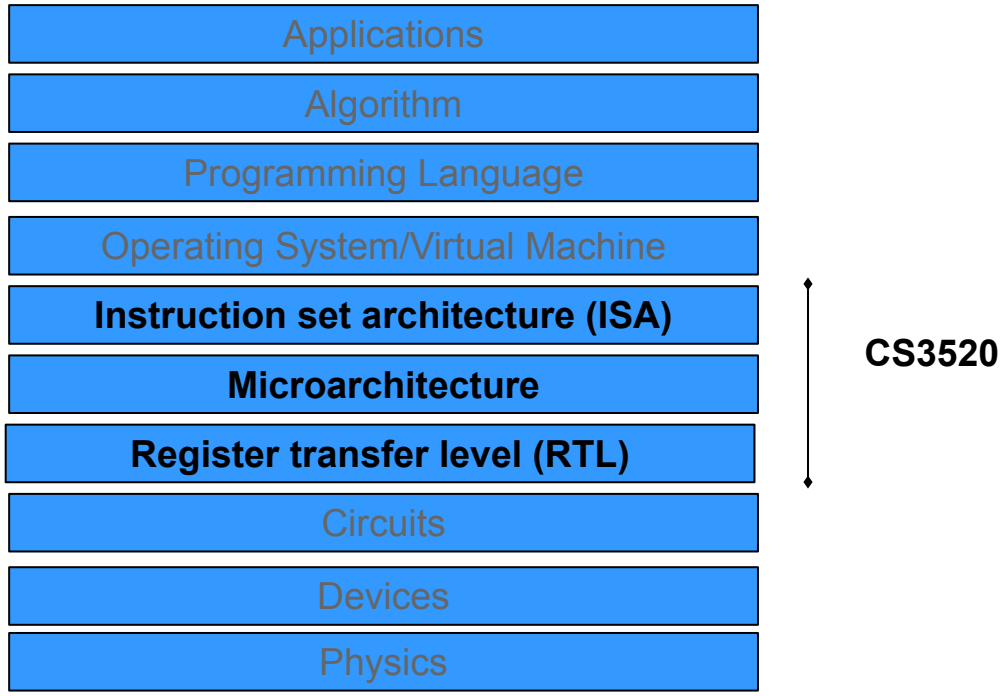
- Brief overview of CS3520 – Computer Organisation and Architecture I
  - Definitions – Computer organisation and architecture
  - Motivations – why study computer organisation and architecture
  - Objectives – what we hope to achieve through CS3520
  - Content – overview of topics to be covered
  - Course handout
- Brief history of computers
- Trends
- Performance issues

BRIEF OVERVIEW OF CS3520:
Computer Organisation and Architecture I

What is computer organisation and architecture?



Definitions – what is computer organisation and architecture?



Definitions – what is computer organisation and architecture?

- **Computer architecture** - computer architecture is a set of rules and methods that describe the functionality, organization, and implementation of computer systems. Some definitions of architecture define it as describing the capabilities and programming model of a computer but not a particular implementation. It is a designer's view of the computer.
- **Computer organisation** - a computer's organization expresses the realization of the architecture or how features are implemented like these registers ,those data paths or this connection to memory. Typical components of a computer organisation are ALU, CPU and memory. Concerned with the user (programmer)'s view of the computer system.

Definitions – what is computer organisation and architecture?

- Design and organisation of a computer system including all sub-systems
  - Design typically begins with the **instruction set** that the processor will execute
  - Includes design of the **arithmetic and logic** hardware elements that carry out computations
  - Design of the **register set** for holding operands for computations
  - Design of the **control unit** that supervises the execution of instructions
  - Design of the internal **buses** that facilitate inter-communication between the sub-systems
  - Design of the **I/O** sub-system that enable interfacing with the outside world

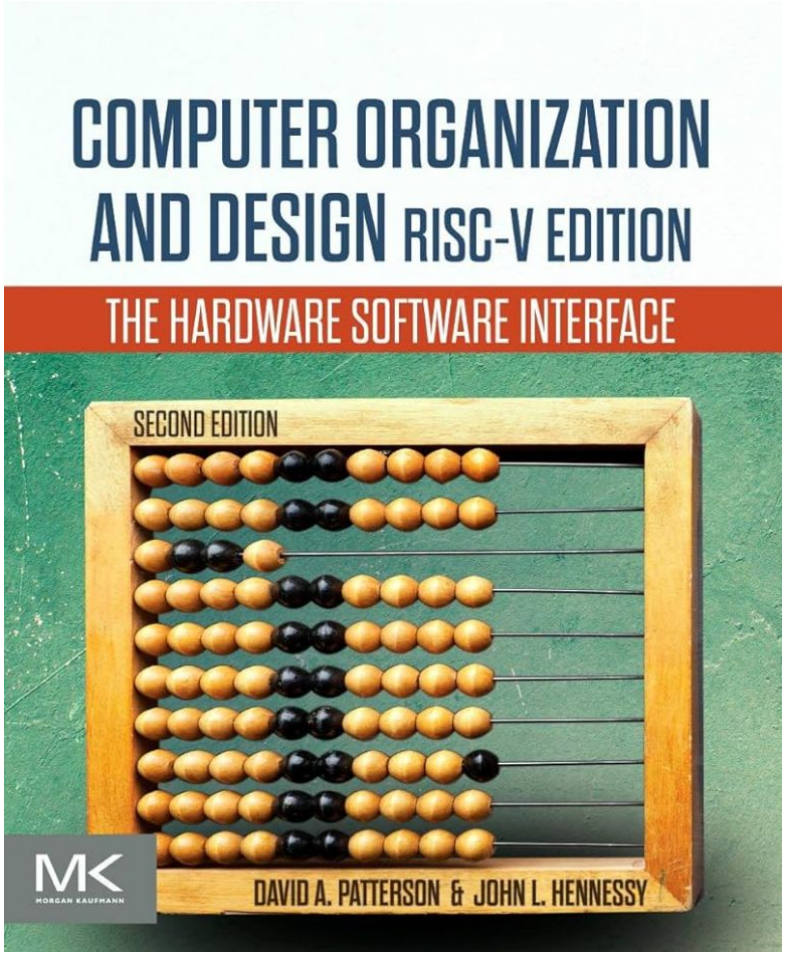
CS3520 – Course Description

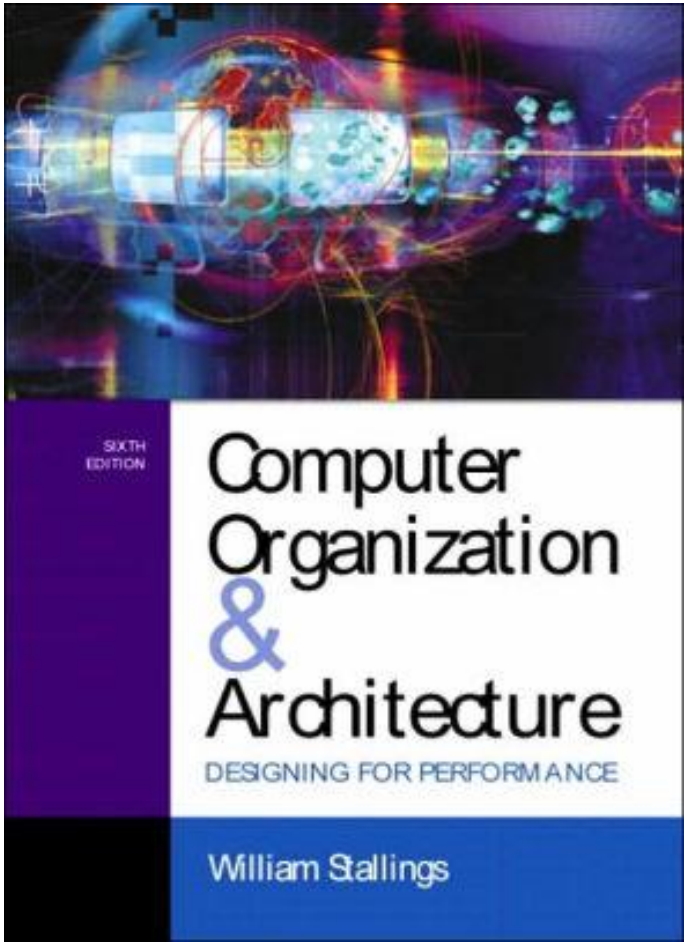
This course provides a comprehensive overview of computer organization and architecture, focusing on how computers execute programs and manage data. Major topics include instruction set architectures (with examples such as RISC-V and x86), assembly language programming, processor design (data path and control, including an introduction to pipelining), arithmetic and numeric representations, memory hierarchy (caches and virtual memory), storage systems, input/output mechanisms, and an overview of parallel architectures. Students will gain both theoretical understanding and practical skills (through assembly programming and design exercises) that prepare them to analyze and design basic computing systems. By the end of the course, students will be equipped to understand the hardware-software interface and how architectural decisions impact system performance.

CS3520 – Course Content

1. **Introduction to Computer Systems:** Overview of computer architecture, von Neumann model, basic components and their functions.
2. **Instruction Set Principles:** Machine instructions, operands, and operations. RISC vs CISC philosophies, example architectures (RISC-V instruction set basics and an overview of x86).
3. **Assembly Language Programming (RISC-V):** Registers, addressing modes, instruction formats. Writing simple assembly programs, procedure calls, stack management, and I/O in assembly.
4. **Data Representation and Computer Arithmetic:** Binary numbers, two's complement, integer arithmetic overflow, floating-point representation (IEEE 754) and arithmetic.
5. **CPU Organization and Design:** CPU data path components (ALU, registers, control unit), control flow, instruction cycle (fetch/decode/execute). Designing a single-cycle CPU for a subset of RISC-V instructions.
6. **Performance and Pipelining:** Basic performance metrics (CPU clock, CPI, etc.). Introduction to pipelining, pipeline stages, pipeline hazards (structural, data, control) and hazard mitigation (stalling, forwarding, branch prediction). Pipeline vs. non-pipeline performance.
7. **Memory Hierarchy:** Memory organization, caches (cache mapping techniques, cache misses and hit rates, cache optimization strategies), main memory (RAM types), introduction to virtual memory (pages, page tables, TLB). How memory hierarchy impacts performance.
8. **Storage Systems:** Secondary storage technologies (magnetic disks vs SSDs), storage hierarchy, RAID basics. Interaction between CPU, memory, and storage (e.g., how data is fetched from disk to memory).
9. **Input/Output Systems:** I/O fundamentals, memory-mapped I/O vs isolated I/O, interrupts mechanism and direct memory access (DMA). Examples of peripheral interfaces.
10. **Parallel and Advanced Architectures:** Overview of multi-core processors, concept of threads and concurrency at hardware level, Flynn's taxonomy (SISD, SIMD, MIMD), brief look at GPUs and their parallel architecture for data-parallel tasks. Trends in modern architectures (power efficiency, introduction to RISC-V as a modern ISA).

Teaching and Learning Resources - Books

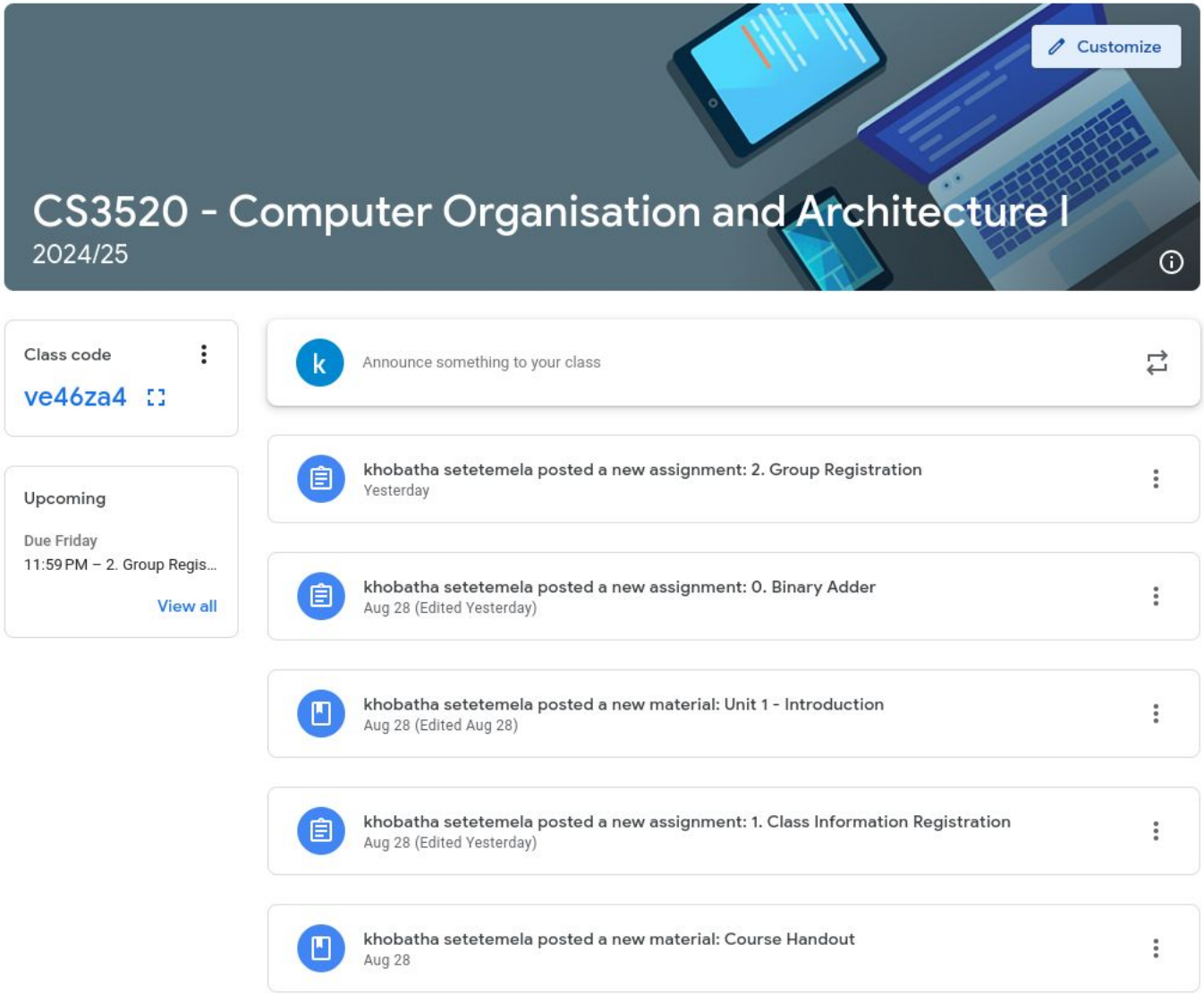




Teaching and Learning Resources - Thuto



Teaching and Learning Resources - Google Classroom



Teaching and Learning Resources – Reading Assignments



Assessment – Class Tests

- Class tests will be administered to test the student's theoretical and conceptual understanding of the course
- The tentative class test schedule is as follows:
  - Test 1 – Week 4
  - Test 2 – Week 8

Assessment – Lab Tests

- Lab tests will be administered to test the ability of the students to solve given Computer Organisation and Architecture problems given limited time
- The tests will be based on the concepts and skills covered during the lecture and lab sessions
- The tests are closed book
- The tentative lab tests schedule is as follows
  - Practical Test 1 – Week 5
  - Practical Test 2 – Week 9

Assessment – Assignments

- Practical assignments will be given from time to time to allow the students to practice the practical aspects of the course during the course of the semester
- Kindly check the course handout and Thuto for the tentative assignment schedule

Assignment No.	Scope
1	Assembly programming
2	CPU datapath design
3	CPU control design
4	CPU pipeline design
5	Computer memory design
6	Computer I/O design

Assessment – Technology Seminars

- The course will include student presentations on various relevant topics in Computer Organisation and Architecture
- The aim is to introduce students to principles of research and professional presentation in information technology and also enhance their teamwork skills
- Students should form **seminar facilitation groups (SFGs)** by Friday 29th **August 2025**
- Technology topics will be assigned to each facilitation group as a reading assignment
- Seminar schedule will be prepared and communicated via Thuto and Classroom
- Guidelines on the format and marking rubric for the seminar presentations will also be communicated

Assessment – Design Project

- The students will have an opportunity to practice the principles and skills learned in this course through a mini design project
- The project will be done in phases and each phase will require the students to apply a part of the skills learned in the course to a basic Computer Organisation and Architecture design problem
- The project will be done in groups – students are required to register their design project groups by **Friday 29th August 2025**
- Students will be required to prepare a final report and demonstrate their final design work at the end of the course

Overall Course Assessment

CATEGORY	WEIGHTING
Tests	10%
Quizzes	10%
Assignments	5%
Project	10%
Other*	5%
Final Exam	60%

*seminars and presentations

Course Policies

- Late submissions
  - 1 day 25% penalty
  - 2 days 50% penalty
  - 3 days 75% penalty
  - >4 days 100% penalty
- Plagiarism
  - Any form of plagiarism is strongly discouraged.
  - Plagiarism punishable by zero and further University disciplinary measures will be taken.



CS3520
Computer Organisation and Architecture I
Lecture 1 – Introduction: Historical Overview

Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: khobatha.setetemela@gmail.com

Lecture Outline

- Brief overview of CS3520 – Computer Organisation and Architecture I
  - Definitions – Computer organisation and architecture
  - Motivations – why study computer organisation and architecture
  - Objectives – what we hope to achieve through CS3520
  - Content – overview of topics to be covered
  - Course handout
- Brief history of computers
- Trends
- Performance issues

BRIEF HISTORY OF COMPUTERS:

The First Computers







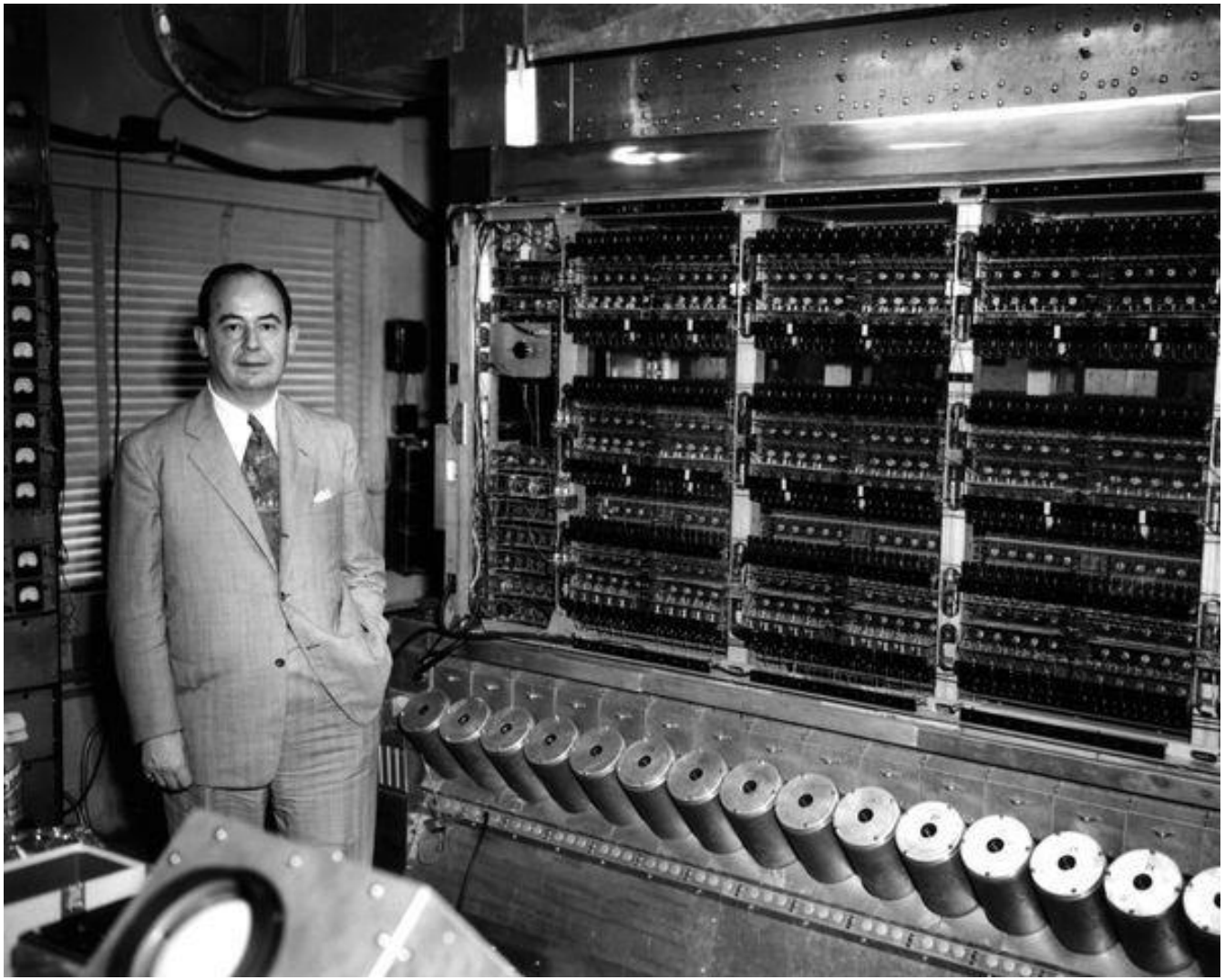
The First Computers

- **Human Computers:**
  - Originally, "computers" were people, not machines.
  - Employed to perform complex mathematical calculations by hand or with simple mechanical tools.
- **Historical Context:**
  - **17th to 19th Century:** Human computers were crucial in fields like astronomy, engineering, and navigation.
  - **World War II:**
    - Surge in demand for human computers, especially for calculating ballistic trajectories and codebreaking.
    - Many women were employed as computers in military and scientific projects.
- **Transition to Machines:**
  - Mid-20th century saw the development of electronic computers (e.g., ENIAC).
  - Machines began automating tasks previously done by human computers.
- **Legacy:**
  - Human computers laid the foundation for modern computing.
  - Contributions were vital to the development of computing and technology.

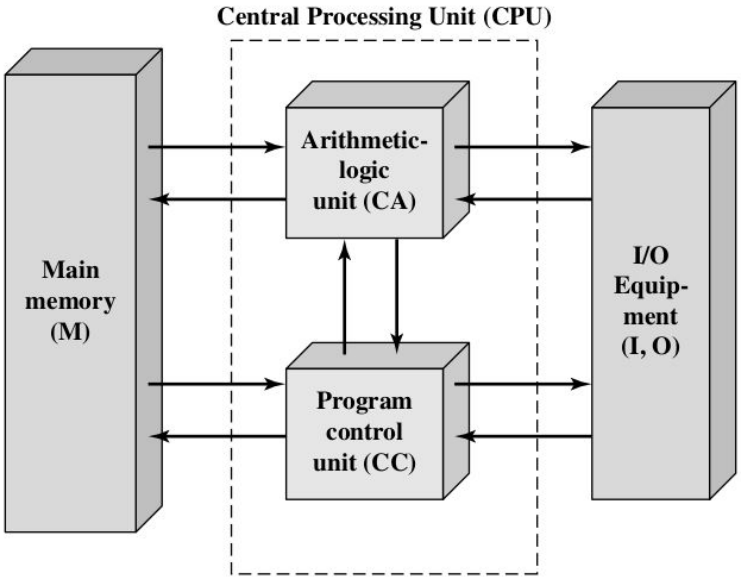
Computer Generations

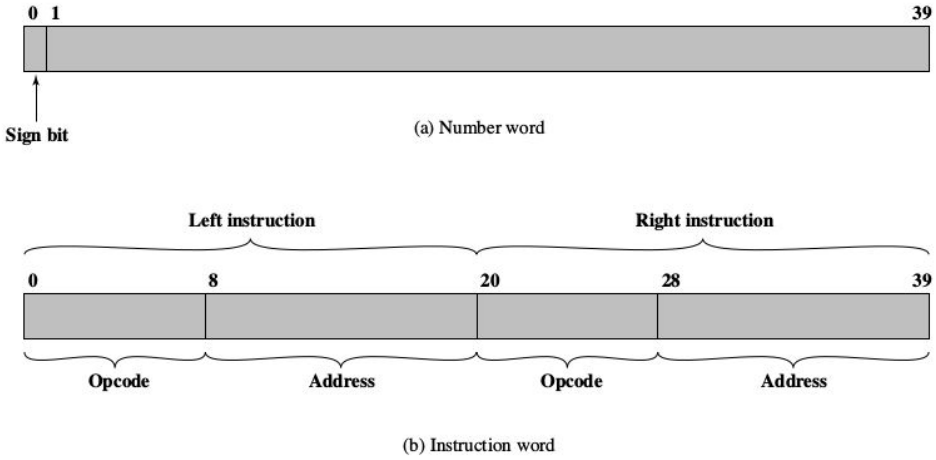
generation	period	technology	software and hardware	examples
1	1945-54	Vacuum tubes and relay memories	Machine language, Von Neumann architecture	EDVAC, UNIVAC, ENIAC, IAS
2	1955-64	Transistors	Assembly programming, general purpose registers, virtual memory, indirect addressing, HW interrupts, batch processing	IBM 7094, Atlas
3	1965-74	Medium-scale integration (MSI); Pipelining; Cache;	High-level programming, Time-sharing OS, parallel processing, first mini-computers, work-stations, super-computing	PDP-8, IBM 370
4	1975-90	VLSI, vectorisation, multicomputers	Multiprocessor OS, compiler environments for parallel processing, VLSI RAM/ROM, first micro-computers	Sun SPARC, Cray X-MP, Intel 486
5	1991-2005	ULSI	Massively parallel processing, pervasive networking,	Cray MPP,
6	2006-2019	ULSI	Parallel computing, cloud computing, big data analytics, GPGPU, 4IR, Hadoop	Intel Core i7, GPGPU
7	2020 -Present	4IR Technologies (AI, IoT, Blockchain, Quantum Computing)	AI/ML frameworks, Edge computing, Blockchain platforms, Quantum computing, Advanced Robotics, Autonomous systems	NVIDIA Jetson, IBM Q, Google Tensor Processing Unit (TPU), Amazon Web Services (AWS), Tesla Autopilot

Generation I Example: IAS



Generation I Example: IAS Organisation and Architecture





➔ **CPU:**

- **Instruction Set:** 20-bit binary instructions with an opcode and address part.
- **Instruction Cycle:** Fetch - Decode - Execute

➔ **Memory:**

- **Size:** 1024 memory locations.
- **Word Length:** 40-bit words, each storing either two 20-bit instructions or one 40-bit data value.
- **Organization:** Single memory space for both instructions and data.

Class Exercise

Consider the IAS Organisation and Architecture and answer the following questions:

(1) How many instructions can be stored in the entire IAS memory of 1024 locations?

(2) How many bits are needed to uniquely address each memory location?

(3) In a 20-bit instruction, if 6 bits are used for the opcode, how many bits are left for the address part?

(4) What is the maximum number of unique opcodes and memory addresses that can be represented?

Brief Overview of the IAS Organisation and Architecture:

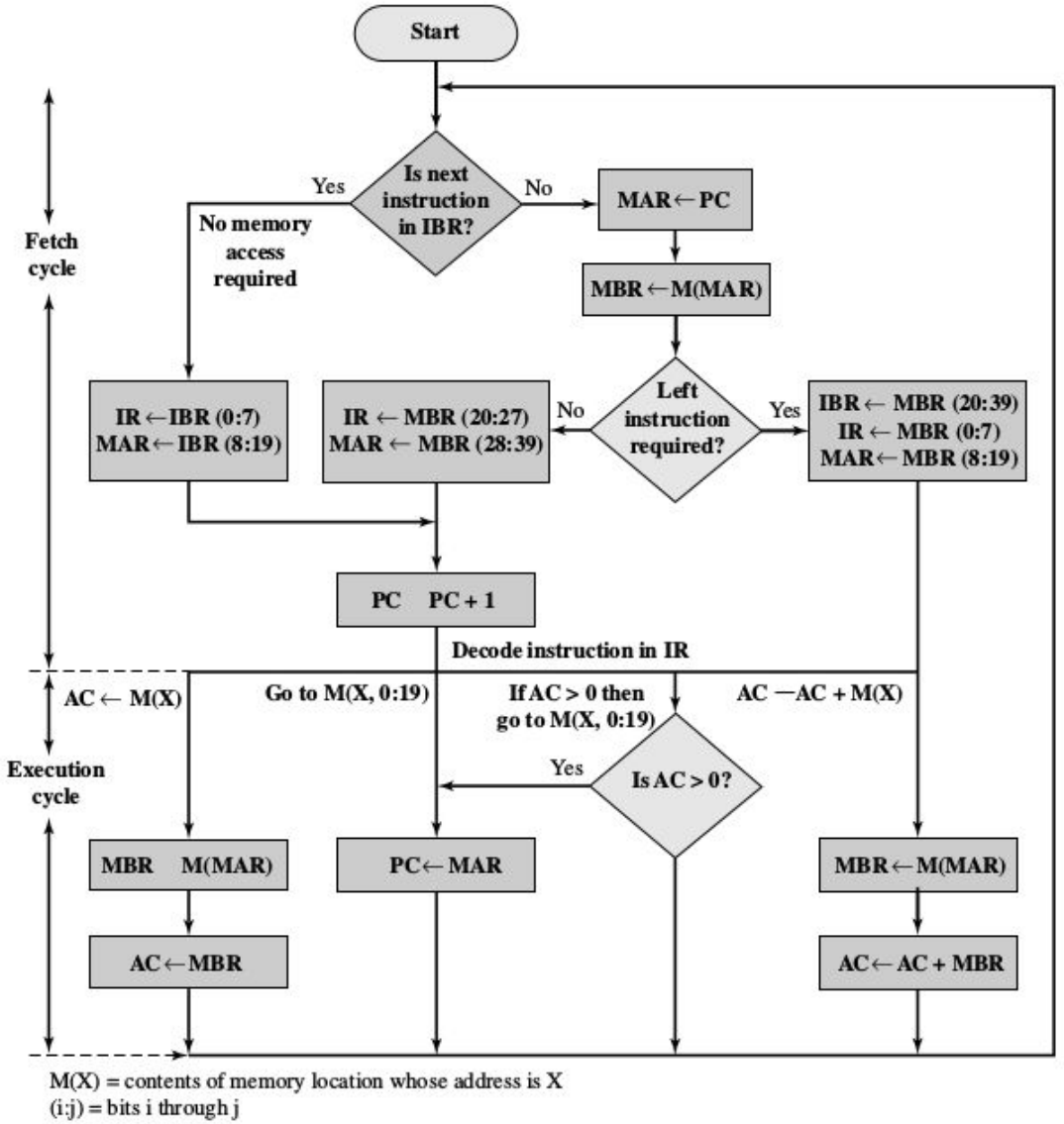
- **CPU:**
  - **Instruction Set:** 20-bit binary instructions with an opcode and address part.
  - **Instruction Cycle:** Fetch - Decode - Execute
- **Memory:**
  - **Size:** 1024 memory locations.
  - **Word Length:** 40-bit words, each storing either two 20-bit instructions or one 40-bit data value.
  - **Organization:** Single memory space for both instructions and data.

Generation I Example: IAS Instruction Set

Instruction Type	Opcode	Symbolic Representation	Description
Data transfer	00001010	LOAD MQ	Transfer contents of register MQ to the accumulator AC
	00001001	LOAD MQ,M(X)	Transfer contents of memory location X to MQ
	00100001	STOR M(X)	Transfer contents of accumulator to memory location X
	00000001	LOAD M(X)	Transfer M(X) to the accumulator
	00000010	LOAD −M(X)	Transfer −M(X) to the accumulator
	00000011	LOAD |M(X)|	Transfer absolute value of M(X) to the accumulator
	00000100	LOAD −|M(X)|	Transfer −|M(X)| to the accumulator
Unconditional branch	00001101	JUMP M(X,0:19)	Take next instruction from left half of M(X)
	00001110	JUMP M(X,20:39)	Take next instruction from right half of M(X)
Conditional branch	00001111	JUMP+ M(X,0:19)	If number in the accumulator is nonnegative, take next instruction from left half of M(X)
	00010000	JUMP+ M(X,20:39)	If number in the accumulator is nonnegative, take next instruction from right half of M(X)
Arithmetic	00000101	ADD M(X)	Add M(X) to AC; put the result in AC
	00000111	ADD |M(X)|	Add |M(X)| to AC; put the result in AC
	00000110	SUB M(X)	Subtract M(X) from AC; put the result in AC
	00001000	SUB |M(X)|	Subtract |M(X)| from AC; put the remainder in AC
	00001011	MUL M(X)	Multiply M(X) by MQ; put most significant bits of result in AC, put least significant bits in MQ
	00001100	DIV M(X)	Divide AC by M(X); put the quotient in MQ and the remainder in AC
	00010100	LSH	Multiply accumulator by 2; i.e., shift left one bit position
	00010101	RSH	Divide accumulator by 2; i.e., shift right one position
Address modify	00010010	STOR M(X,8:19)	Replace left address field at M(X) by 12 rightmost bits of AC
	00010011	STOR M(X,28:39)	Replace right address field at M(X) by 12 rightmost bits of AC

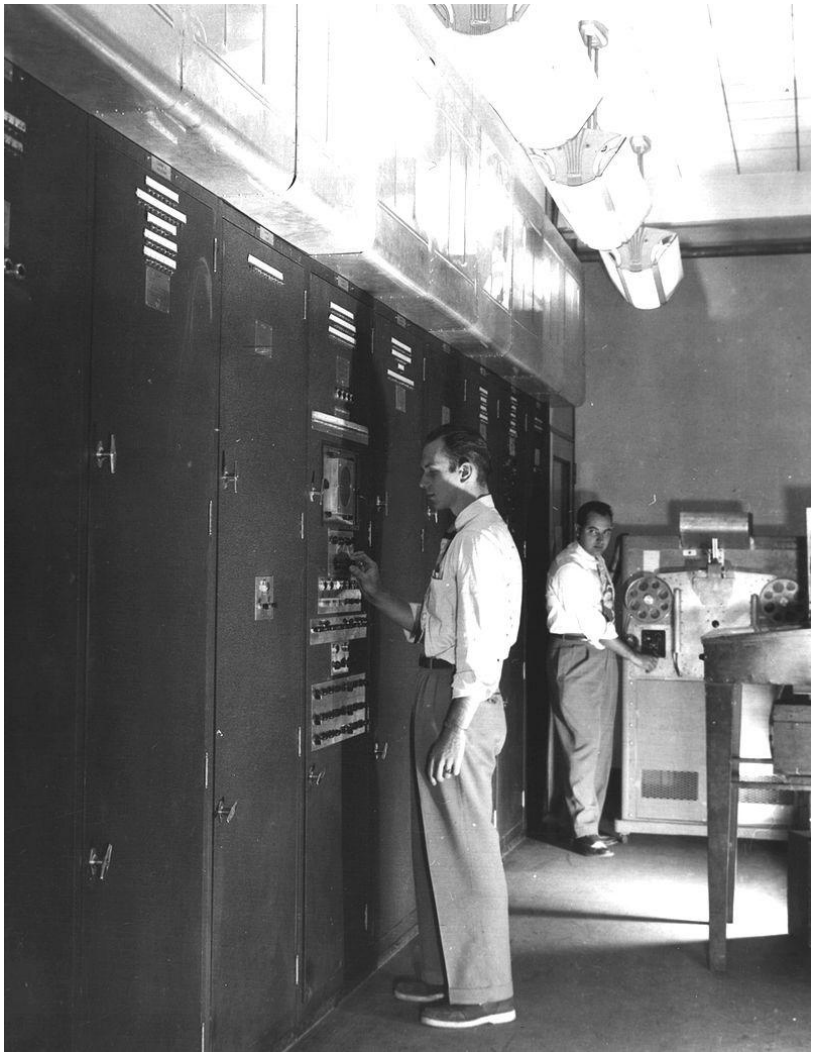
Generation I Example: IAS Instruction Cycle

- **IBR:** Instruction Buffer Register
- **MBR:** Memory Buffer Register
- **MAR:** Memory Address Register
- **PC:** Program Counter
- **IR:** Instruction Register
- **AC:** Accumulator
- **MQ:** Multiplier Quotient Register



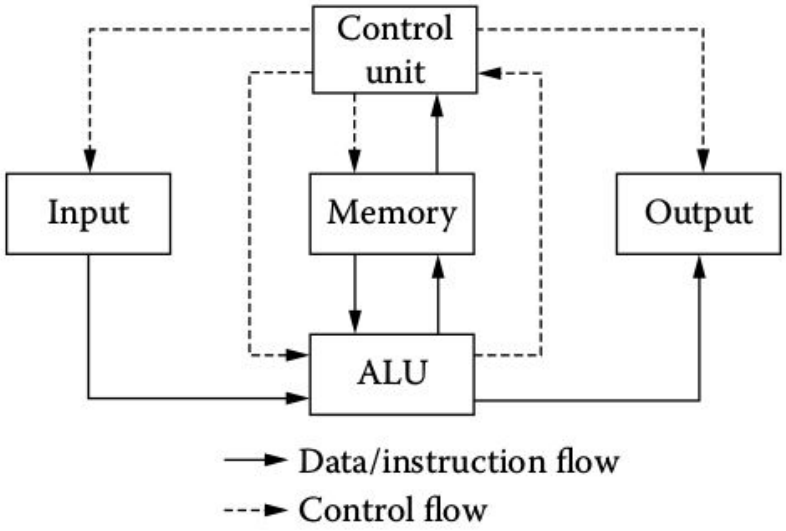
Generation I Example: EDVAC

- First modern computer to run a program stored in memory
- Designed by Von Neumann and team – designed to be a stored-program computer
- Used binary system
- Embodied the concept now known as the Von Neumann architecture



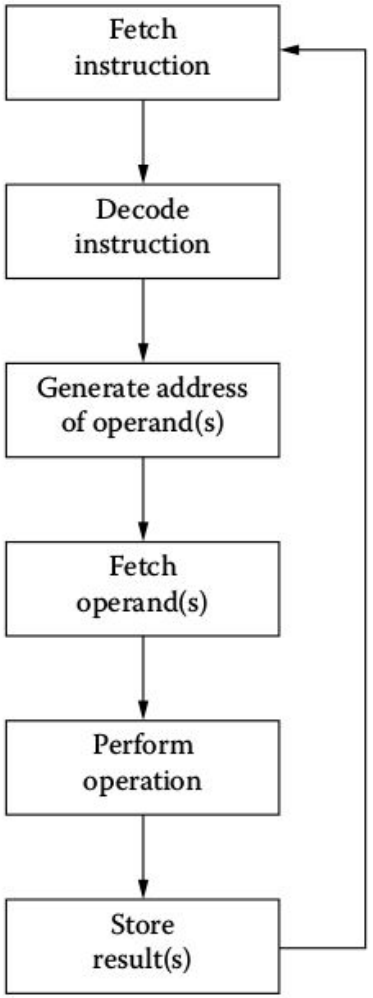
Generation I: Von Neumann Architecture

- First modern design for a computer based on the **stored program concept** originally developed by Babbage
- Forms the **basis of most modern computer systems**
- **Single memory** to store both data and program instructions
- **Control unit** and **ALU** key components of the architecture
- **Internal registers** added later to provide faster storage for a limited amount of data
- Original application domain was numerical solution of scientific and engineering problems involving differential equations
- But now lends itself to many other application domains!
- Stored-program concept the secret of success!



Generation I: Von Neumann Machine Cycle

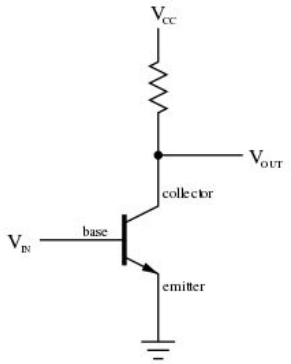
- **Fetch instruction** - CPU begins by fetching the instruction by reading it from memory
- **Decode instruction** - Instruction is then decoded i.e t CPU hardware interprets the instruction bits and determines what operation needs to be done, the addresses of the operands are also determined
- **Fetch operands** - The operands are then fetched fror registers, memory locations or an input device and se to the ALU
- **Execute** - the requested operation is performed
- **Store results** - Finally the results are stored in the specified location
- We will discuss and examine the details of the hardware and software required to carry out the Von Neumann machine cycle later in this course

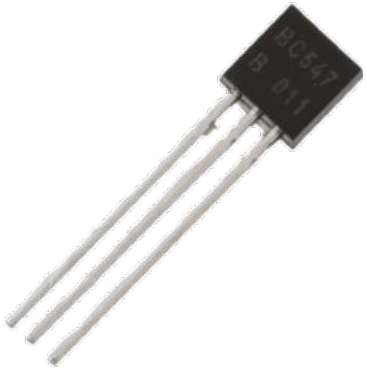


Generation II: The Birth of the Transistor

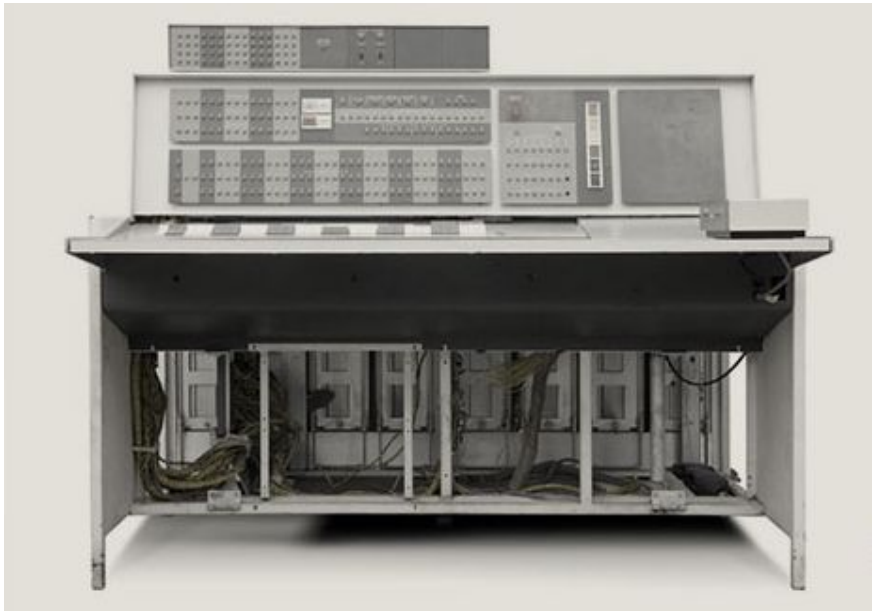
- **Semiconductor device** used to amplify or switch electronic signals and electronic power
- It is composed of semiconductor material usually with **at least three terminals** for connection to an external circuit
- A voltage or current applied to one pair of the transistor's terminals controls the current through another pair of terminals.
- Because the controlled (output) power can be higher than the controlling (input) power, a transistor can amplify a signal.
- **Transistors revolutionized the field of electronics**, and paved the way for **smaller** and **cheaper** radios, calculators, and computers, among other things.
- Compared with the vacuum tube, transistors are generally **smaller**, require **less power** to operate and have higher MTBF (114 - 1140 years)

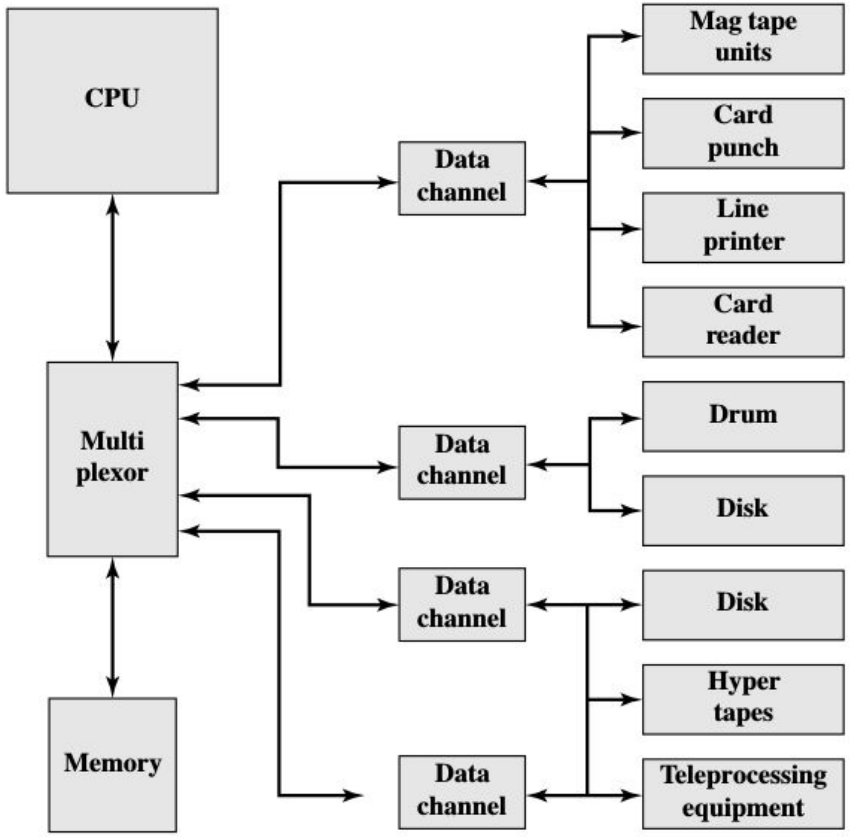




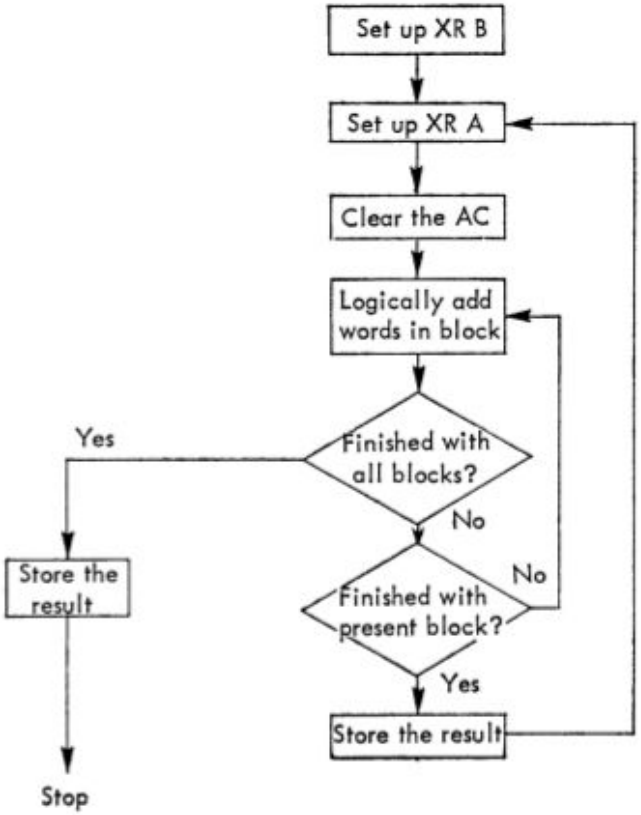


Generation II Example: IBM 7094 Organisation



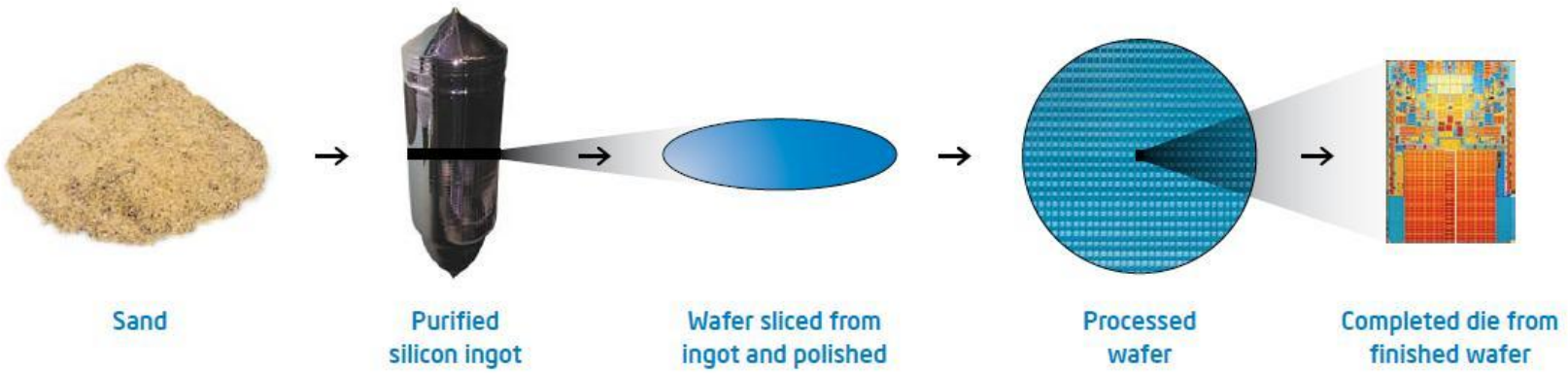


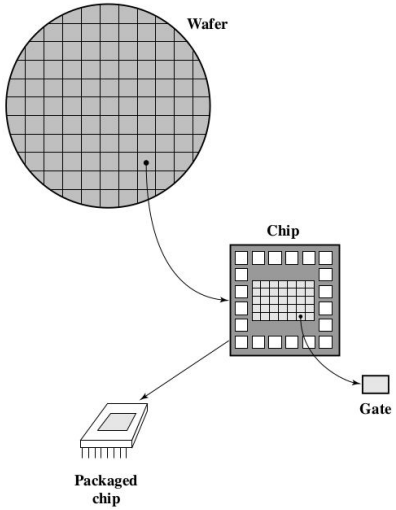
Generation II Example: IBM 7094 Assembly Programming

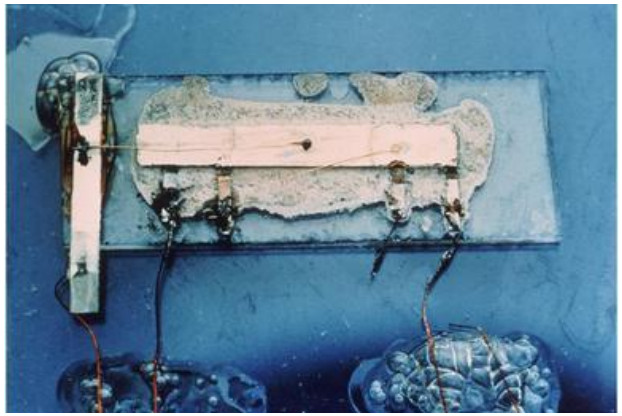


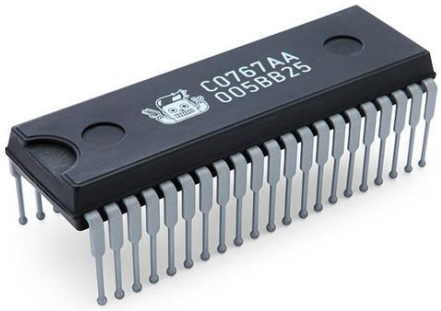
H	LOCATION		OPERATION	ADDRESS, TAG, DECREMENT/COUNT	
1	2 6	7	8		
	0100		LXA	501,B	49 to XRB
	0101		LXA	500,A	9 to XRA
	0102		CLM		Clear the accumulator
	0103		ACL	650,B	Add the block
	0104		TNX	110,B,1	Test all blocks for end
	0105		TIX	103,A,1	Reduce count
	0106		SLW	640,B	Store the check sum for block
	0107		TIX	101,B,1	Test for end of block
	0110		SLW	640	Store check sum (last one)
	0111		HPR		Stop

Generation III: The Making of an Integrated Circuit









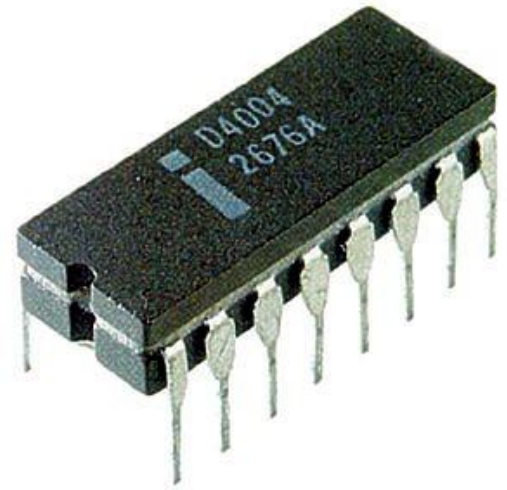
Generation III Example: PDP-8

- The PDP-8 is a **12-bit** minicomputer that was produced by Digital Equipment Corporation (DEC).
- **First commercially successful minicomputer**, with over 50,000 examples being sold over the model's lifetime.
- Introduced on 22 March 1965 priced at $18,500 (equivalent to about $150,000 in 2018)
- Using a one-bit serial arithmetic logic unit (ALU) allowed the PDP-8/S to be **smaller and less expensive**
- The greatest historical significance was that the PDP-8's **low cost and high volume** made a computer **available to many new customers** for many new uses
- Most **machine code** was **generated by compilers and report generators**
- The **reduced instruction set computer (RISC)** returned full-circle to the PDP-8's emphasis on a simple instruction set and achieving multiple actions in a single instruction cycle, in order to maximize execution speed, although the newer computers have much longer instruction words.



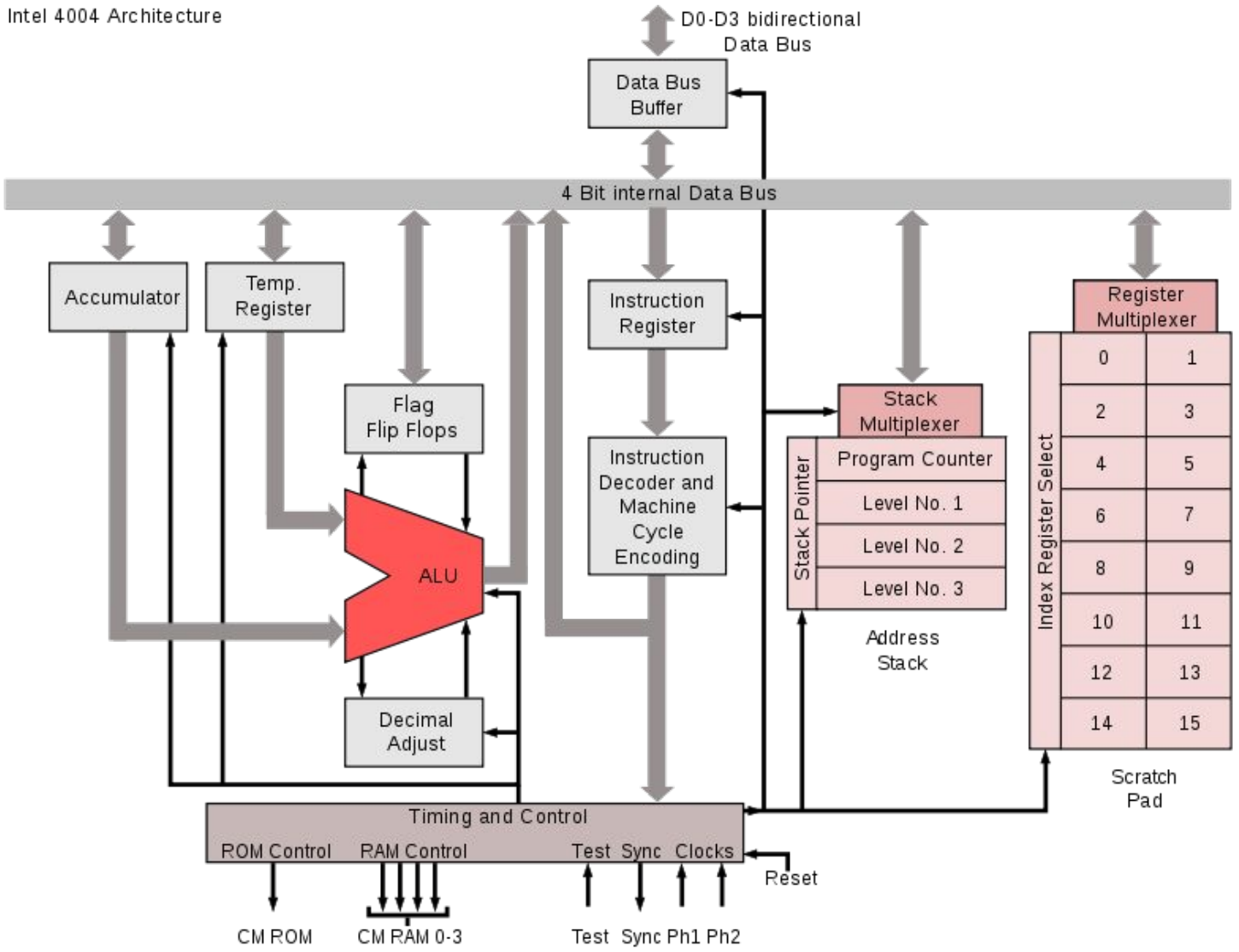
Generation IV: The Birth of Microprocessors

- A microprocessor is a computer processor that **incorporates the functions of a central processing unit on a single integrated circuit (IC),** or at most a few integrated circuits.
- The microprocessor is a **multipurpose**, **clock driven**, **register based**, digital integrated circuit that accepts binary data as input, processes it according to instructions stored in its memory and provides results as output.
- Microprocessors contain both **combinational logic** and **sequential digital logic**.
- The integration of a whole CPU onto a single or a few integrated circuits greatly **reduced the cost of processing power.**
- The **Intel 4004** is generally regarded as the first commercially available microprocessor.





Generation IV Example: Intel 4004 Architecture



Generation IV Examples



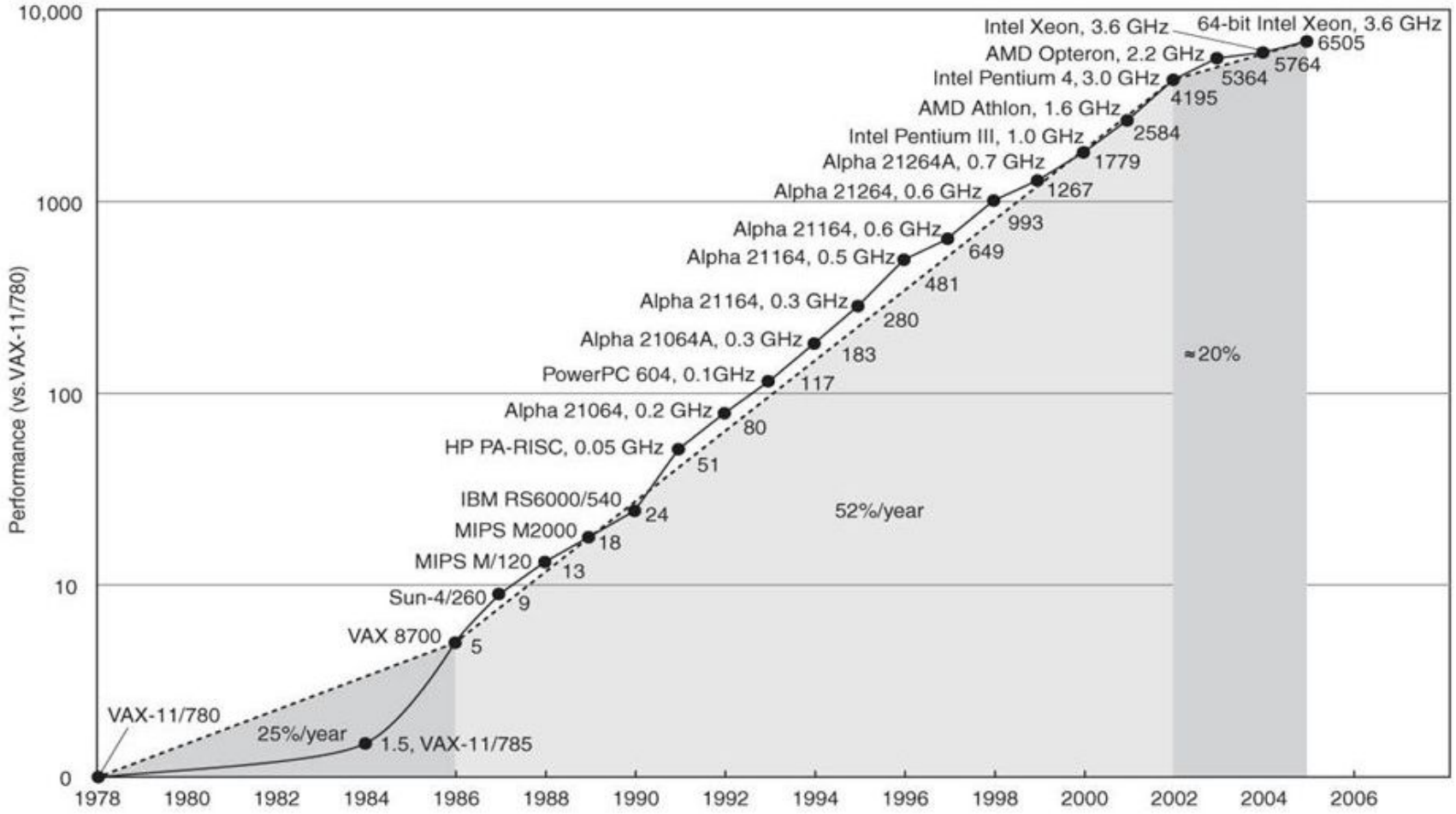


Generation V: More Integration

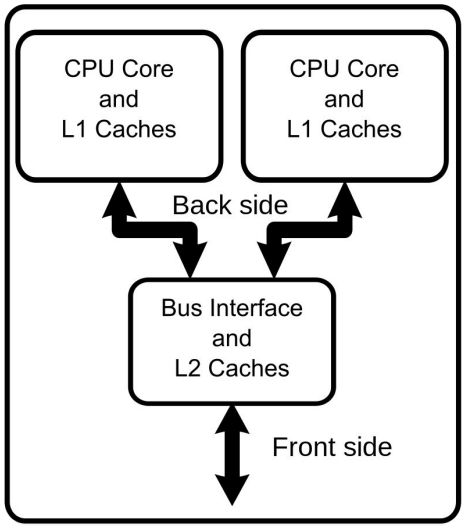




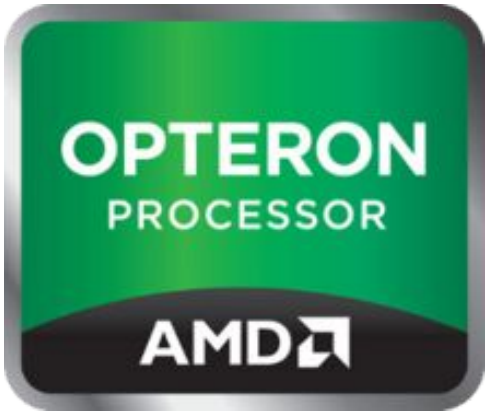
Generation V: Hitting the Wall



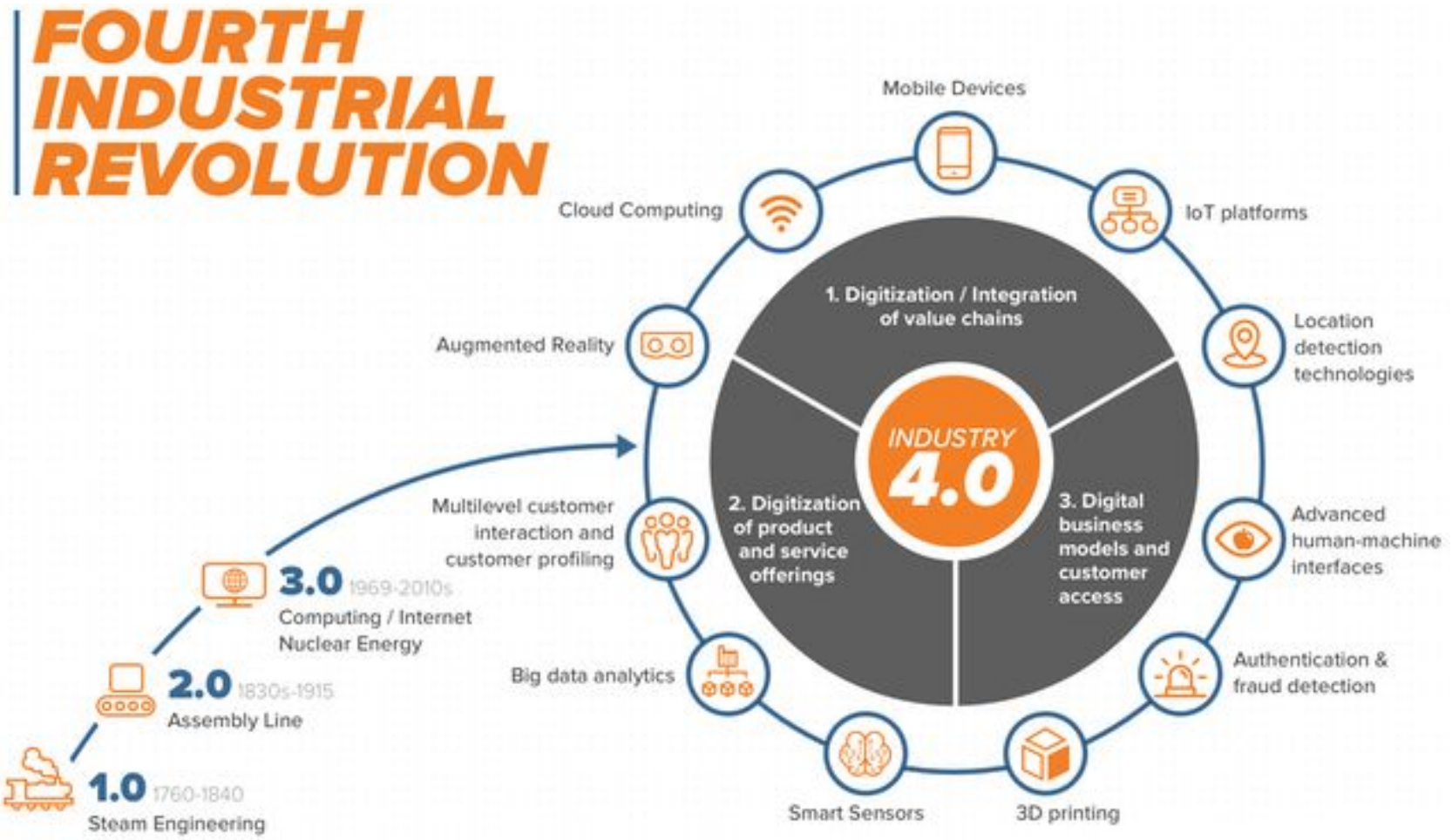
Generation V: The Birth of the Multicore Era







Generation VI: Modern Computing





CS3520
Computer Organisation and Architecture I
Lecture 2 – Introduction: Trends and Performance Issues

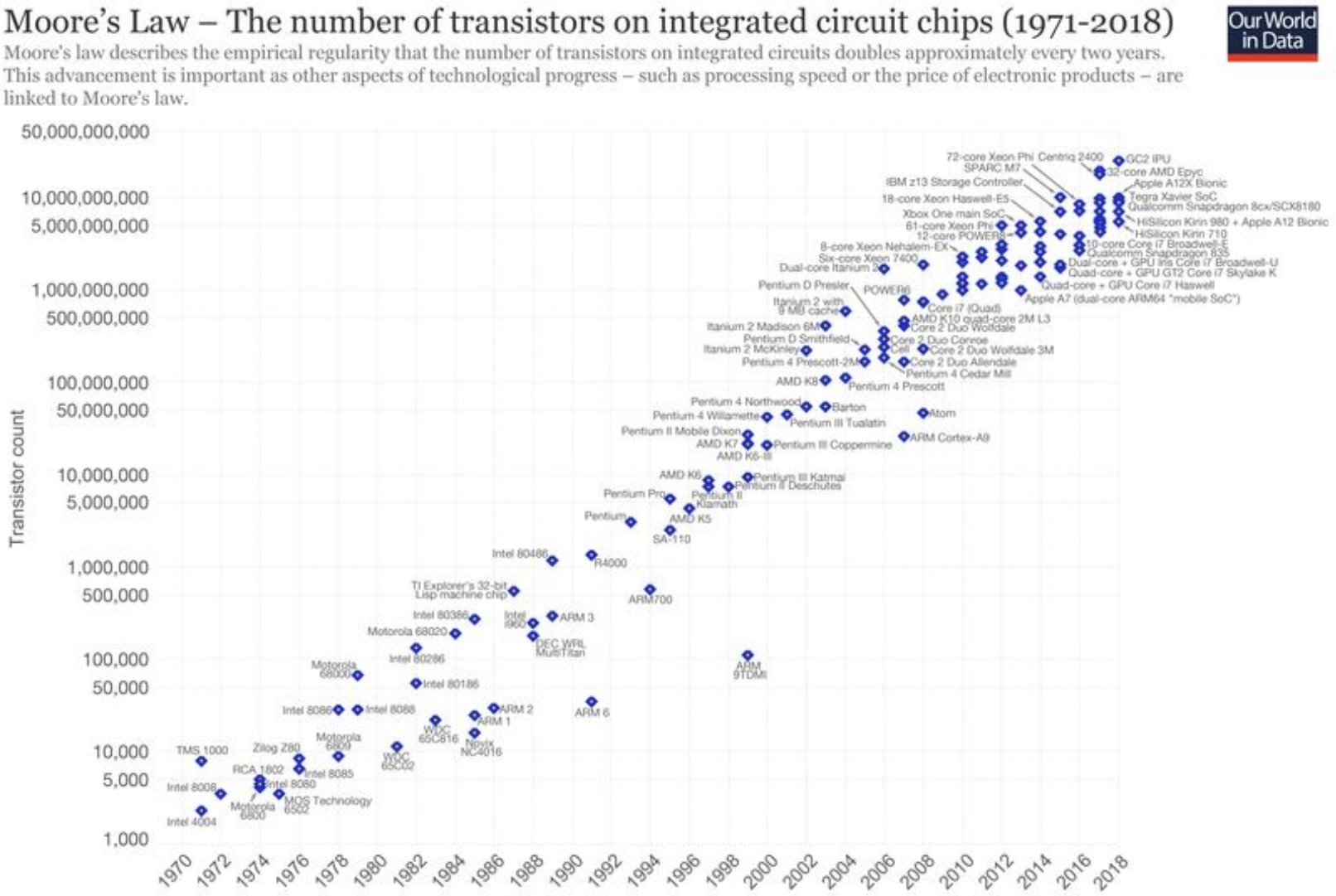
Mr K.O, Setetemela
BJO 308
Department of Mathematics & Computer Science
National University of Lesotho
w: 59105441 | e: khobatha.setetemela@gmail.com

Lecture Outline

- Brief overview of CS3520 – Computer Organisation and Architecture I
  - Definitions – Computer organisation and architecture
  - Motivations – why study computer organisation and architecture
  - Objectives – what we hope to achieve through CS3520
  - Content – overview of topics to be covered
  - Course handout
- Brief history of computers
- Trends
- Performance issues

TRENDS:
Technology, Power and Cost

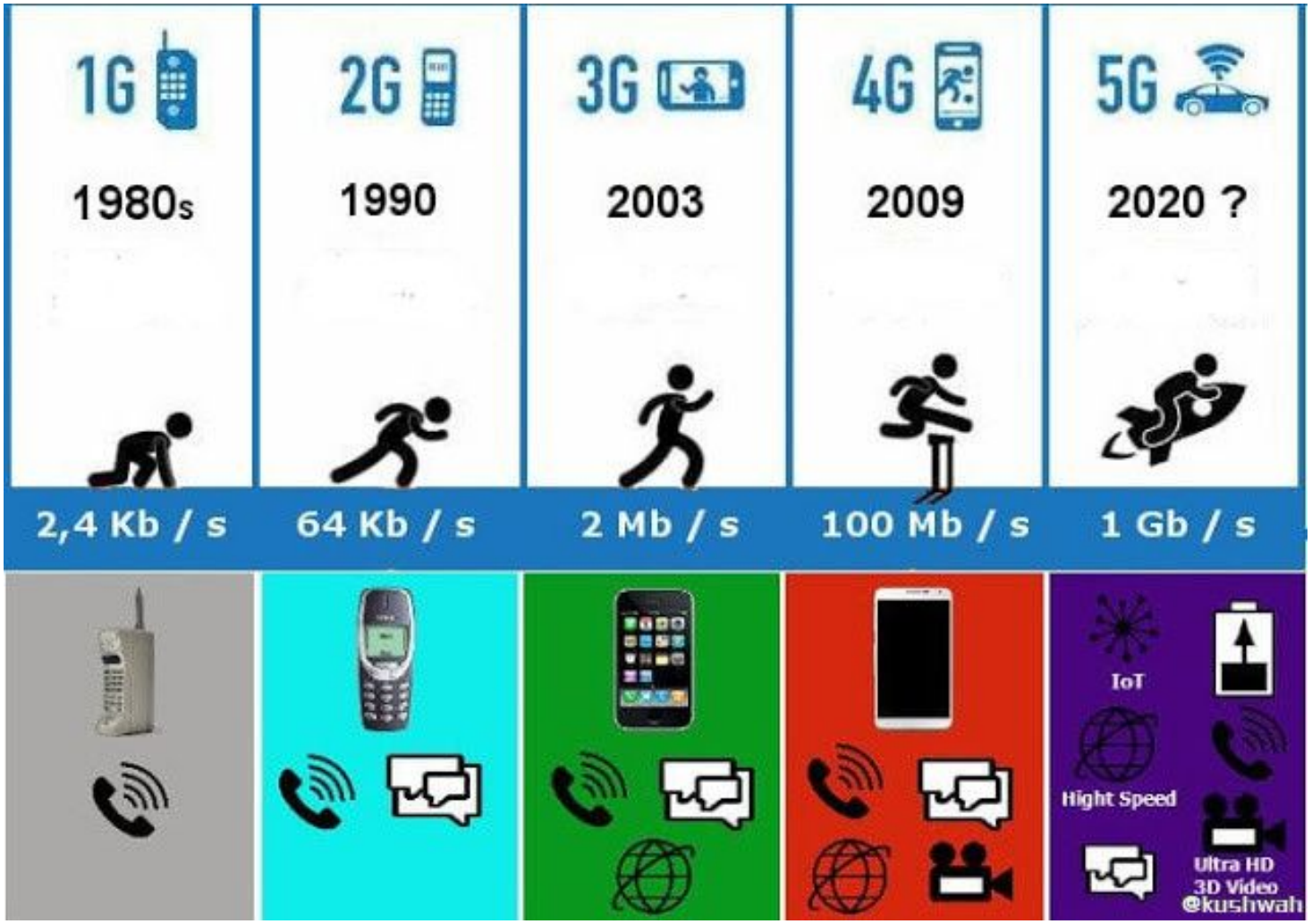
Trends in Technology – Moore's Law



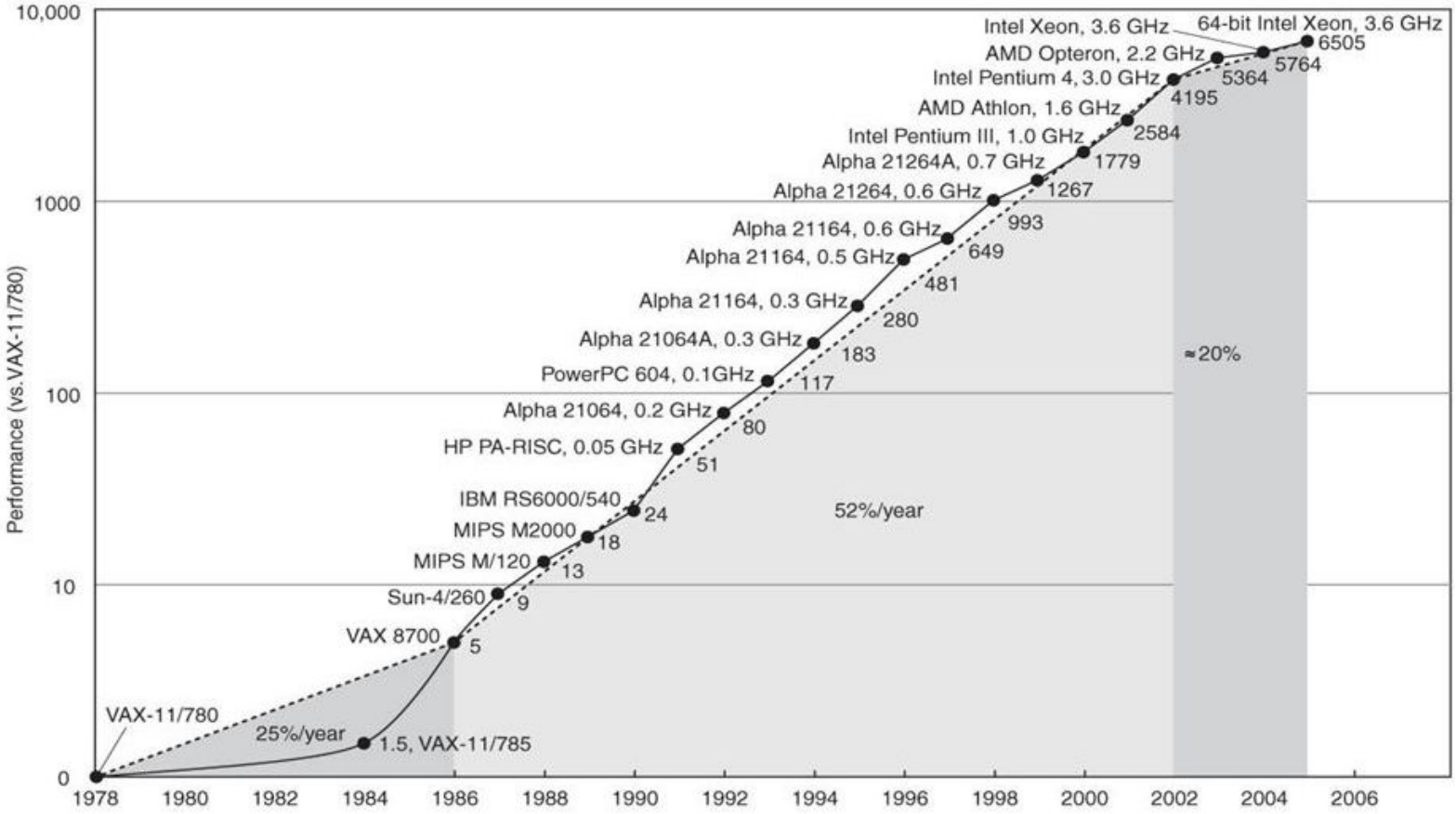
Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)
The data visualization is available at OurWorldinData.org. There you find more visualizations and research on this topic.

Licensed under CC-BY-SA by the author Max Roser.

Trends in Technology – Networking

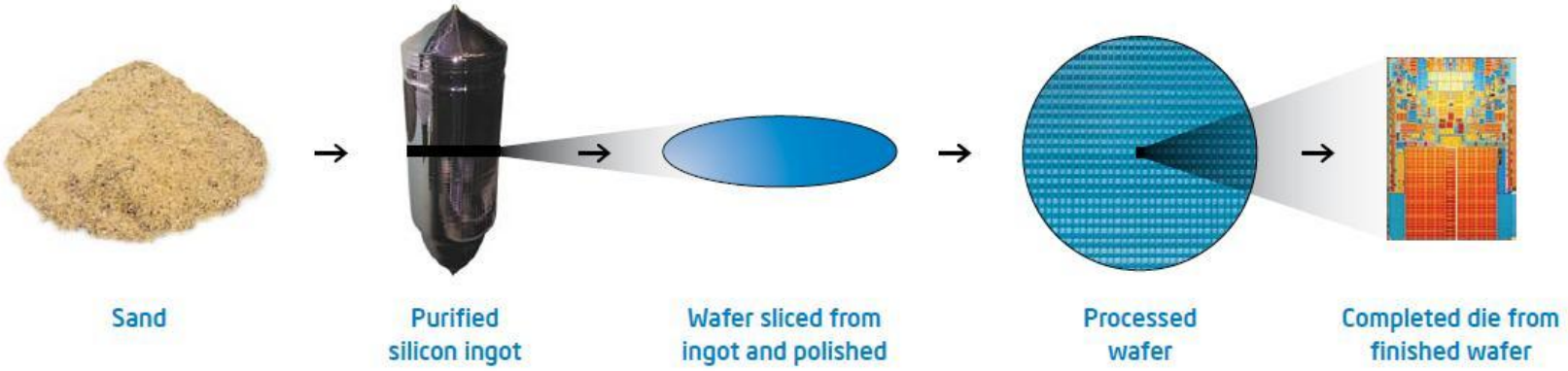


Trends in Power and Energy – The Power Wall

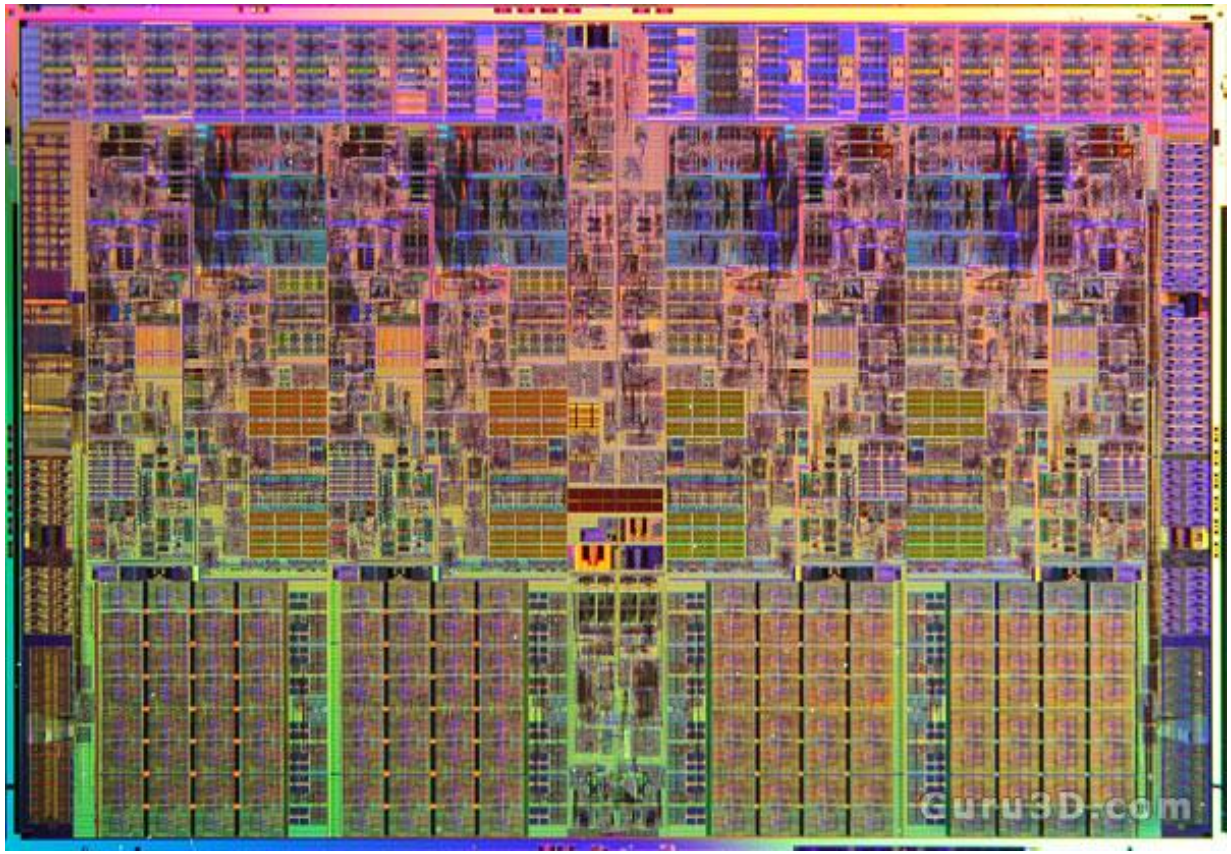


Trends in Cost

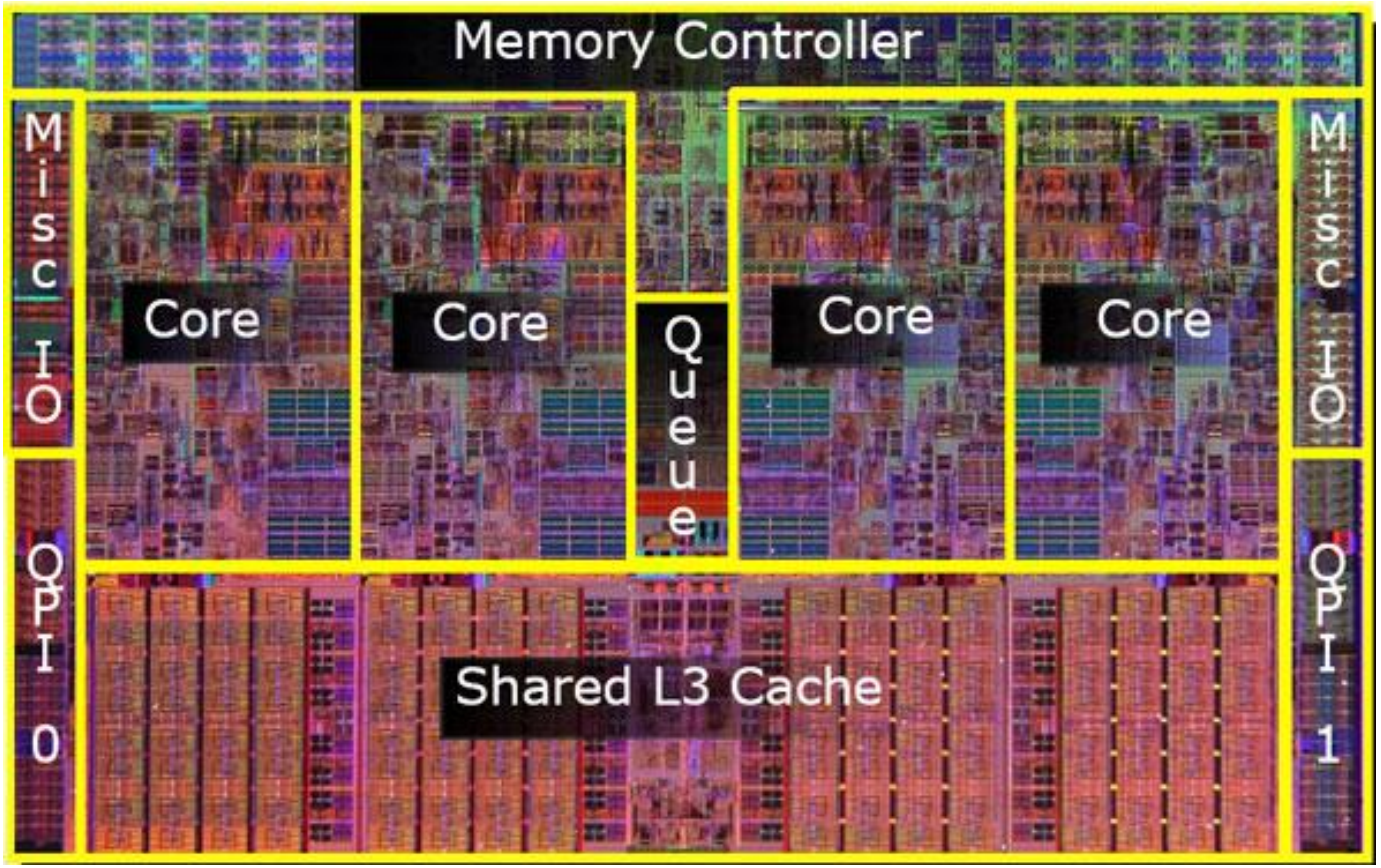
- The use of technology improvements to lower cost, as well as increase performance, has been a major theme in the computer industry
- Computer designers must understand the cost of chips to understand the cost of current computers
- Although the cost of ICs have dropped drastically, the basic process of silicon manufacture has remained the same: a wafer is still tested and chopped into dies that are packaged



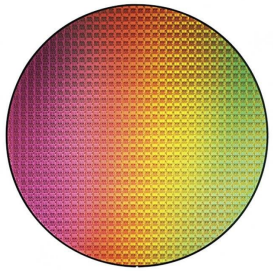
Photograph of an Intel Core i7 CPU die



Photograph of an Intel Core i7 CPU die



Manufacturing Cost

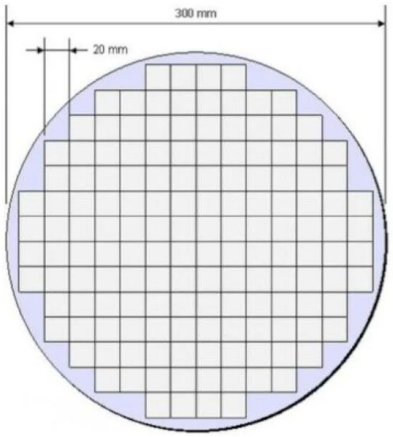


- **Cost of an IC**

  = (Cost of die + Cost of testing die + Cost of packaging and final test) ／ Final test yield

- **Cost of die**

  = Cost of wafer ／ (Dies per wafer x Die yield)



- **Dies per wafer**

  = π x (Wafer diameter/2)^2 ／ Die area - π x Wafer diameter ／ $\sqrt{}$(2 x Die area)

Cost versus Price

- **Commoditization of computers**: The gap between the cost of manufacturing a computer and its selling price is decreasing.
- **Shrinking profit margins**: Reduced margins are due to increased competition and mass production.
- **R&D spending:**
  - Commodity PC business: typically spends 4% of income on R&D, which includes all engineering efforts.
  - High-end server business: spends up to 12% of income on R&D.
- **Surprising for engineers**: Many engineers are unaware that only a small percentage of revenue is dedicated to R&D.

Cost of Manufacturing versus Cost of Operation

- **Warehouse-scale computers**:
  - contain tens of thousands of servers.
  - making the cost of operation a significant factor alongside the cost of purchase.
- **Amortized purchase price**:
  - Servers and networks represent over 60% of the monthly cost of operating a warehouse-scale computer.
  - This assumes a short lifetime for IT equipment (3-4 years).
- **Operational costs**:
  - Around 30% of monthly operational costs are attributed to power consumption and infrastructure costs for power distribution and cooling.
  - The infrastructure costs are amortized over a longer period (10 years).
- **Energy efficiency**: To reduce operational costs, architects need to focus on designing energy-efficient systems.

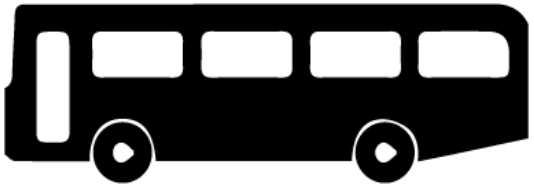


PERFORMANCE ISSUES:
Measuring, reporting and summarising

The Difficulty of Performance Benchmarking: Which One is Faster?







What is Performance?

- When we say one computing platform has better performance than the other, what do we mean?
- As seen from the cars analogy
  - In order to best compare the cars in terms of performance, we would need to first define what we mean by performance
  - We also need to define what performance means in computing
- Several different definitions of performance
  - **Desktop computing:** a faster program is the one that gets the job done first? That is, shorter **response time**.
  - **Mobile computing:** a better performing mobile device is the one that lets me enjoy long battery life?
  - **Cluster computing:** the faster server is the one that gets most jobs completed in a day? That is, higher **throughput** and **bandwidth**.
- Therefore, in most cases different benchmark kernel applications and performance metrics are needed to benchmark different computing platforms
  - Generally, **response time** used for personal computing platforms such as mobile devices
  - **Throughput** and **bandwidth** used for large scale computing platforms such as servers
  - **Both** response time and throughput used for high-performance embedded computing

Performance Measurement: User View

- Time is generally the most basic measure of computer performance
  - The computer that performs the same amount of work in the shortest time is the fastest
- Program execution time measurement
  - Wall clock time / response time/ elapsed time
  - Includes disk accesses, memory accesses, I/O operations, OS overhead etc
- CPU time
  - Time spent by the CPU processing a task
  - Excludes I/O overhead, disk accesses, memory accesses
  - CPU time = User CPU time + System CPU time

Performance Measurement: Designer View

- Computer users and designers often consider performance using different metrics
  - Users concerned about response time, throughput
  - Designers concerned about clock speed, dynamic power,
  - word length etc
- By relating these different metrics, we could determine the effect of a change in the computer design aspects on performance as experienced by the user

Program CPU Execution Time (sec) = No. of CPU Clock Cycles for Program x Clock Cycle Time
= Program instructions count x Average Clock Time per Instruction (CPI)
= Program instructions count x CPI x Clock Cycle Time (sec)

Exercise: Throughput and Response

Do the following changes to computer system increase throughput, decrease response time, or both? Motivate.

(1) Replacing processor in a computer with a faster version
(2) Adding additional processors to a system that uses multiple processors separate tasks - for e.g searching the web

Exercise: Throughput and Response

1. Suppose we know that an application that uses both personal mobile devices and the Cloud is limited by network performance. For the following changes, state whether only the throughput improves, both response time and throughput improve, or neither improves.

(a) An extra network channel is added between the PMD and the Cloud,increasing the total network throughput and reducing the delay to obtain network access (since there are now two channels).
(b) The networking software is improved, thereby reducing the network communication delay, but not increasing throughput.
(c) More memory is added to the computer.

2. Computer C's performance is 4 times as fast as the performance of computer B, which runs a given application in 28 seconds. How long will computer C take to run that application?

Exercise: Improving Performance

Our favorite program runs in 10 seconds on computer A, which has a 2GHz clock. We are trying to help a computer designer build a computer B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program.

Q:What clock rate should we tell the designer to target?

Exercise: Improving Performance (Solution)

Our favorite program runs in 10 seconds on computer A, which has a 2GHz clock. We are trying to help a computer designer build a computer B, which will run this program in 6 seconds. The designer has determined that a substantial increase in the clock rate is possible, but this increase will affect the rest of the CPU design, causing computer B to require 1.2 times as many clock cycles as computer A for this program.

Q: What clock rate should we tell the designer to target?

Ans:

CPU_B = No. of CPU Clock Cycles for Prog B x CCT_B

6 = 1.2 x (No. of CPU Clock Cycles for Prog A) x CCT_B

Where:

No. of CPU Clock Cycles for Prog A = CPU_A/CCT_A

$= 6/(1/2x10^9) = 20x10^9$ cycles

Therefore: $CCT_B = 6/(1.2 \times 20x10^9)$

$\Rightarrow f_B = 1/CCT_B = 4GHZ$

Take Home Exercises: Improving Performance

1. One computer system has a 3.2-GHz processor, and another has only a 2.7-GHz processor. Is it possible that the second system might outperform the first? Explain.

2. Consider three different processors P1, P2, and P3 executing the same instruction set. P1 has a 3 GHz clock rate and a CPI of 1.5. P2 has a 2.5 GHz clock rate and a CPI of 1.0. P3 has a 4.0 GHz clock rate and has a CPI of 2.2.

2.1 Which processor has the highest performance expressed in instructions per second?
2.2 If the processors each execute a program in 10 seconds, find the number of cycles and the number of instructions.
2.3 We are trying to reduce the execution time by 30% but this leads to an increase of 20% in the CPI. What clock rate should we have to get this time reduction?